

Projet de Recherche et Développement
EN IoT AND POST-QUANTUM ENCRYPTION

FILIERE: IoT AND BIG DATA

IoT Edge Computing + Post-Quantum Encryption

Réalisé par :

M. Zounoogo Oussama Mahomet KOURITA

Encadré par:

Pr. Omar SOUISSI INPT



AGENCE NATIONAL DE RÉGLEMENTATION DES TÉLÉCOMMUNICATIONS

INSTITUT NATIONAL DES POSTES ET TÉLÉCOMMUNICATIONS

Promotion : 2024/2025

Remerciements

Ce projet de recherche et de développement a été rendu possible grâce à la contribution et au soutien de plusieurs personnes et institutions. Je tiens à exprimer ma profonde gratitude à :

- **Pr. Omar SOUISSI**, de l'INPT, pour son encadrement précieux, ses conseils avisés et son soutien tout au long de la réalisation de ce travail. Son expertise a été une source d'inspiration et de guidance constante.
- **L'Institut National des Postes et des Télécommunications (INPT)** et **l'Agence Nationale de Réglementation des Télécommunications (ANRT)**, pour avoir fourni un cadre de recherche exceptionnel et les ressources nécessaires pour mener à bien ce projet.

Leur engagement a été essentiel au succès de cette étude sur la cryptographie post-quantique et son application dans l'Internet des Objets.

Résumé

Ce rapport de R&D détaille la conception et l'implémentation d'une infrastructure de communication sécurisée pour l'Internet des Objets (IoT), en combinant des algorithmes de cryptographie classiques et post-quantiques (PQC). L'architecture du système repose sur trois composants principaux : des capteurs simulés, une passerelle de périphérie (**Edge Gateway**) et un **Serveur Central**.

L'infrastructure de sécurité est fondée sur un double jeu d'algorithmes de cryptographie. Un script Python, `generate_keys.py`, gère la création et le stockage des clés pour les méthodes classiques (**RSA** pour le chiffrement et **ECDSA** pour la signature) et post-quantiques (**ML-KEM-768/Kyber** pour l'encapsulation de clés et **ML-DSA-65/Dilithium** pour la signature).

La simulation des capteurs génère des données de température, d'humidité, de GPS et de pression, qu'elle publie sur un broker MQTT en continu ou en mode ponctuel.

L'**Edge Gateway** joue un rôle crucial en collectant ces données via MQTT, en appliquant une couche de chiffrement hybride pour chaque message, et en mesurant les performances de ces opérations (latence, utilisation CPU/mémoire et taille des messages).

Le Serveur Central reçoit ces messages, les déchiffre en utilisant les clés appropriées, et stocke les métriques de performance dans une base de données **SQLite**. Il offre également une interface web interactive en **HTML/JavaScript** qui fournit une visualisation en temps réel de ces métriques via des graphiques **Chart.js**. L'interface permet de comparer directement les performances des méthodes de chiffrement classiques et post-quantiques, et de contrôler à distance les simulations et le gateway.

Ce projet établit une solution robuste pour la sécurité des données IoT, capable de faire face aux menaces actuelles et aux défis posés par l'informatique quantique. Les résultats de la recherche démontrent un compromis entre l'efficacité actuelle des algorithmes classiques et la résilience future des algorithmes PQC, posant les bases pour une stratégie de migration progressive.

List of Figures

5.1	Interface Principale	11
5.2	Graphiques de Performance (Latence, CPU, Taille)	12
5.3	Tableau Récapitulatif et Aperçu des Données	13

Contents

Remerciements	i
Résumé	ii
List of Figures	iii
Table of Contents	v
1 Introduction	1
2 Simulation de Capteurs IoT	2
2.1 Simulation en Mode Continu	2
2.2 Simulation à Envoi Unique	3
3 Infrastructure Cryptographique et Génération des Clés	4
3.1 Génération des Clés Classiques	4
3.2 Génération des Clés Post-Quantiques (PQC)	5
4 L’Edge Gateway	6
4.1 Collecte et Préparation des Données	6
4.2 Mécanismes de Chiffrement Hybride	6
4.2.1 Chiffrement Classique	6
4.2.2 Chiffrement Post-Quantique (PQC)	7
4.3 Mesure et Transmission des Performances	7
5 Le Serveur Central et l’Interface de Surveillance	9
5.1 Logique du Backend	9
5.1.1 Réception et Déchiffrement des Données	9
5.1.2 Stockage et Accès aux Données	10

5.1.3	Points de Terminaison d'API RESTful	10
5.2	Interface Frontend	11
5.2.1	Contrôle Centralisé	11
5.2.2	Visualisation et Analyse des Performances	11
Conclusion et Observations de la Recherche		14
Glossaire		15
Liste des Acronymes		16

Chapter 1

Introduction

Ce projet de R&D a pour objectif de développer une infrastructure de communication sécurisée pour l'Internet des Objets (IoT), en combinant des technologies existantes avec des solutions de cryptographie post-quantique (PQC) pour anticiper les menaces futures posées par les ordinateurs quantiques. Le système est composé de trois parties principales : des capteurs simulés, une passerelle (Edge Gateway) et un serveur central. Le rapport détaille la conception, l'implémentation et les fonctionnalités de chaque composant, en mettant en évidence l'architecture hybride de sécurité qui utilise à la fois des algorithmes de chiffrement classiques et post-quantiques. L'objectif est de démontrer la faisabilité et de comparer les performances de ces approches cryptographiques dans un environnement IoT réel.

Objectifs du Projet

- Étudier l'impact des algorithmes post-quantiques sur la latence et la mémoire.
- Comparer les performances du chiffrement post-quantique avec celles du chiffrement classique.
- Identifier les contraintes pour une intégration dans des environnements réels.

Chapter 2

Simulation de Capteurs IoT

Cette section décrit la mise en œuvre de deux scripts Python simulant le comportement de capteurs connectés à travers le protocole MQTT. Ces capteurs agissent comme la source de données pour l'ensemble du système, publiant des informations sur un broker MQTT et se distinguant par leur méthode de publication : continue ou à envoi unique.

2.1 Simulation en Mode Continu

Le premier script, conçu pour simuler un flux de données en temps réel, génère et publie des informations à une fréquence régulière.

- **Configuration et Communication** : Le script se connecte au broker MQTT à l'adresse `localhost:1883`. Il utilise la bibliothèque `paho-mqtt` et une *Quality of Service (QoS)* de **1**, assurant que les messages sont livrés au moins une fois.
- **Architecture Multithread** : Pour simuler 30 capteurs en parallèle, le programme dédie un *thread* distinct à chaque capteur. La simulation inclut :
 - **15 capteurs de Température/Humidité** : Génèrent des valeurs aléatoires de température (entre 18.0 et 25.0 °C) et d'humidité (entre 30.0 et 80.0 %). Ils publient sur le topic `sensors/temp_humidity/<id>`.
 - **10 capteurs GPS** : Simulent un mouvement en variant légèrement les coordonnées, démarrant près de Kénitra, Maroc (34.2635°N, -6.5815°W). Ils incluent également la vitesse et l'altitude, avec une publication sur le topic `sensors/gps_tracking/<id>`.

- **5 capteurs de Pression** : Génèrent des valeurs de pression atmosphérique aléatoires (entre 1000.0 et 1030.0 hPa), avec une publication sur le topic `sensors/atmospheric_pressure/<id>`.
- **Format et Fréquence** : Les données sont formatées en **JSON** et envoyées toutes les 5 secondes. La simulation ne s'arrête que par une interruption manuelle (**Ctrl+C**).

2.2 Simulation à Envoi Unique

Ce second script propose une simulation plus simple, où chaque capteur envoie un seul message et se déconnecte immédiatement.

- **Fonctionnement Éphémère** : Chaque capteur se connecte au broker, génère et publie un unique message, puis se termine. Le programme se termine automatiquement une fois que tous les messages ont été envoyés.
- **Message Retenu** : Les messages sont publiés avec le drapeau `retain=True`. Cette option garantit que le broker conserve la dernière valeur reçue sur le topic et la transmet immédiatement à tout nouvel abonné.
- **Différentiation des Topics** : Afin de distinguer les messages de cette simulation, un topic spécifique est utilisé, sous la forme `sensors1fois/<type>/<id>`.

Chapter 3

Infrastructure Cryptographique et Génération des Clés

Ce chapitre décrit en détail la mise en place de l'infrastructure de sécurité hybride, qui est un fondement essentiel de ce projet. Un script Python nommé `generate_keys.py` a été développé pour générer et sauvegarder toutes les paires de clés cryptographiques nécessaires à la communication sécurisée entre l'Edge Gateway et le serveur central.

3.1 Génération des Clés Classiques

Cette section se concentre sur les algorithmes cryptographiques traditionnels, considérés comme standards de l'industrie, en s'appuyant sur la bibliothèque `cryptography`. Les clés générées sont stockées au format **PEM** (Privacy-Enhanced Mail), un format textuel largement utilisé.

- **Côté Passerelle (Gateway) :** Une paire de clés **ECDSA** (Elliptic Curve Digital Signature Algorithm) est générée en utilisant la courbe **SECP256R1**.
 - La **clé privée** est utilisée pour **signer numériquement** les messages. Cette signature permet au serveur de vérifier que les données proviennent bien de la passerelle. Elle est enregistrée sous le nom `gateway_ecdsa_private.pem`.
 - La **clé publique** correspondante est mise à la disposition du serveur pour **vérifier l'authenticité de la signature**. Elle est enregistrée dans le fichier `gateway_ecdsa_public.pem`.
- **Côté Serveur Central :** Une paire de clés **RSA** est générée avec une taille de **2048 bits**.

- La **clé publique** est fournie à la passerelle pour **chiffrer la clé de session AES**. Elle est sauvegardée sous le nom `server_rsa_public.pem`.
- La **clé privée** est conservée par le serveur pour **déchiffrer** la clé de session. Elle est enregistrée dans le fichier `server_rsa_private.pem`.

3.2 Génération des Clés Post-Quantiques (PQC)

Pour se préparer à l'avènement des ordinateurs quantiques, des paires de clés basées sur des algorithmes PQC sont générées. Le script utilise la bibliothèque `pqcrypto`, et les clés sont stockées au format binaire (`.bin`) car elles ne sont pas standardisées pour le format PEM.

- **Côté Passerelle (Gateway)** : Une paire de clés de signature **ML-DSA-65**, connue sous le nom de **Dilithium**, est générée. Cet algorithme, sélectionné par le NIST, est résistant aux attaques quantiques sur la signature numérique. Les clés publique et privée sont respectivement sauvegardées sous `gateway_dilithium_public.bin` et `gateway_dilithium_private.bin`.
- **Côté Serveur Central** : Une paire de clés d'encapsulation de clé **ML-KEM-768**, plus connu sous le nom de **Kyber**, est générée. Cet algorithme, également sélectionné par le NIST, est un *Key Encapsulation Mechanism* (KEM).
 - La **clé publique** est fournie à la passerelle pour **encapsuler** la clé de session symétrique.
 - La **clé privée** est conservée par le serveur pour la **décapsulation** et la récupération sécurisée de cette clé de session.

Le script organise ces clés de manière ordonnée dans un dossier nommé `Keys` pour une gestion et une utilisation ultérieures simplifiées. Cette architecture de clés double assure une robustesse face aux menaces actuelles et futures.

Chapter 4

L'Edge Gateway

L'Edge Gateway est l'intermédiaire central du système, agissant comme un nœud de sécurité et de traitement des données. Son rôle principal est de collecter les données brutes des capteurs via MQTT, d'appliquer une couche de chiffrement hybride, de mesurer les performances de ces opérations, puis de transmettre le tout au serveur central.

4.1 Collecte et Préparation des Données

Le gateway est conçu comme un client MQTT persistant qui s'abonne à tous les topics de capteurs (`sensors/#` et `sensors1fois/#`) avec un Quality of Service (QoS) de 0. Chaque fois qu'un message est reçu, le processus `on_message` est déclenché. Il décode la charge utile (payload) du message JSON reçu des capteurs pour la préparer au chiffrement.

4.2 Mécanismes de Chiffrement Hybride

Pour chaque message reçu, la passerelle exécute deux méthodes de chiffrement distinctes : une classique et une post-quantique. Les clés nécessaires sont chargées depuis le dossier `Keys/`, garantissant une séparation et une gestion sécurisée des identifiants cryptographiques.

4.2.1 Chiffrement Classique

Cette méthode combine un algorithme de chiffrement symétrique robuste avec une cryptographie asymétrique standard.

- **Chiffrement des Données** : Une clé de session aléatoire de 32 octets est générée. Les données

du capteur sont chiffrées à l'aide de l'algorithme **AES-256-GCM**, qui est un mode de chiffrement authentifié assurant à la fois la confidentialité et l'intégrité des données.

- **Encapsulation de la Clé** : La clé de session AES est ensuite chiffrée avec la **clé publique RSA** du serveur central. Cette opération est essentielle pour permettre au serveur de récupérer la clé secrète de manière sécurisée. Le padding **OAEP** est utilisé pour renforcer la sécurité du chiffrement RSA.
- **Signature du Message** : L'intégrité et l'authenticité de la passerelle sont garanties par une signature numérique. La charge utile chiffrée est signée avec la **clé privée ECDSA** de la passerelle, ce qui permet au serveur de vérifier que le message n'a pas été falsifié et qu'il provient d'une source de confiance.

4.2.2 Chiffrement Post-Quantique (PQC)

Cette méthode anticipe la menace des ordinateurs quantiques en utilisant des algorithmes résilients aux attaques quantiques.

- **Encapsulation de la Clé** : Au lieu de RSA, l'Edge Gateway utilise l'algorithme **ML-KEM-768 (Kyber)** pour l'échange de clés. Il génère un texte de chiffrement et une clé de session partagée en utilisant la clé publique **Kyber** du serveur. C'est ce texte de chiffrement qui sera envoyé au serveur pour décapsulation.
- **Chiffrement des Données** : La clé de session issue de l'encapsulation Kyber est utilisée pour chiffrer les données avec **AES-256-GCM**.
- **Signature du Message** : La signature numérique est réalisée avec la **clé privée ML-DSA-65 (Dilithium)** de la passerelle. Cet algorithme PQC de signature est conçu pour résister aux attaques des futurs ordinateurs quantiques.

4.3 Mesure et Transmission des Performances

Pour chaque processus de chiffrement (classique et PQC), le gateway intègre un module de mesure de performance. Avant d'exécuter chaque opération de chiffrement, il capture des métriques clés en utilisant la bibliothèque `psutil`.

- **Latence** : Le temps d'exécution de la fonction de chiffrement, mesuré en millisecondes.

- **Utilisation du Processeur** : Le pourcentage d'utilisation du CPU par le processus du gateway.
- **Utilisation de la Mémoire** : L'empreinte mémoire du processus, mesurée en Méga-octets (Mo).
- **Taille des Données** : Comparaison de la taille de la charge utile originale (à l'entrée) et de la taille du message chiffré (incluant les métadonnées cryptographiques).

Toutes ces métriques sont agrégées avec les deux versions chiffrées du message (classique et PQC) dans un dictionnaire **JSON** unique. Ce payload final est publié sur un topic dédié, `gateway/all_data`, à destination du serveur central. Cette méthode permet non seulement de transmettre les données sécurisées mais également de fournir des éléments de comparaison pour évaluer les coûts de performance de chaque mécanisme de chiffrement.

Chapter 5

Le Serveur Central et l'Interface de Surveillance

Le serveur central est le cerveau de l'architecture, le point de convergence où les données sécurisées et les métriques de performance sont collectées, traitées et visualisées. Il est composé d'une logique backend robuste, construite en Python avec le framework **Flask**, et d'une interface frontend interactive, développée en **HTML/JavaScript** avec les bibliothèques **Tailwind CSS** et **Chart.js**. Contrairement au gateway, le serveur est une entité passive qui attend les messages et gère les requêtes des utilisateurs.

5.1 Logique du Backend

Le backend est conçu pour être à la fois réactif et performant. Il opère avec une architecture multithreadée : un thread est dédié aux communications MQTT, tandis que le thread principal gère l'application web Flask, garantissant que l'interface utilisateur ne se fige pas pendant le traitement des messages.

5.1.1 Réception et Déchiffrement des Données

Le serveur central fonctionne comme un client MQTT qui s'abonne au topic unique `gateway/all_data`. Pour chaque message reçu, il effectue un processus de déchiffrement inversé, validant l'authenticité et l'intégrité des données avant de les traiter.

- **Déchiffrement Classique** : Le serveur utilise sa **clé privée RSA** pour déchiffrer la clé de session AES-256-GCM encapsulée par le gateway. Une fois la clé de session récupérée, il peut déchiffrer les

données elles-mêmes. La **clé publique ECDSA** du gateway est utilisée pour vérifier la signature du message, confirmant qu'il n'a pas été altéré et qu'il provient bien de la passerelle de confiance.

- **Déchiffrement Post-Quantique (PQC)** : Dans ce cas, le serveur utilise sa **clé privée ML-KEM-768 (Kyber)** pour décapsuler le texte de chiffrement et reconstituer la clé de session AES. Il vérifie ensuite la signature avec la **clé publique ML-DSA-65 (Dilithium)** du gateway, garantissant l'authenticité du message dans un contexte post-quantique.

5.1.2 Stockage et Accès aux Données

Une fois les données déchiffrées, la logique du backend les traite de deux manières :

- **Stockage des Métriques** : Les métriques de performance (latence, utilisation CPU, mémoire, et taille des messages) pour chaque type de chiffrement sont stockées de manière persistante dans une base de données **SQLite**. Cela permet de construire un historique de performance et d'effectuer des analyses comparatives sur le long terme.
- **Mise à Jour en Temps Réel** : Les dernières données de capteurs déchiffrées sont conservées dans une variable globale, protégée par un verrou de thread (**threading.Lock**), pour garantir la cohérence des données dans un environnement multithread. Ces données sont ensuite mises à la disposition du frontend via un point de terminaison d'API.

5.1.3 Points de Terminaison d'API RESTful

Le backend expose plusieurs points de terminaison REST qui permettent au frontend de communiquer et d'interagir avec le système :

- **/api/metrics** : Récupère les 50 dernières mesures de performance stockées dans la base de données.
- **/api/data** : Récupère les dernières données de capteurs déchiffrées (classiques et PQC).
- **/api/simulation/*** et **/api/gateway/*** : Ces endpoints sont uniques et confèrent au serveur une capacité de contrôle à distance sur les processus de simulation. En envoyant des requêtes POST, l'interface utilisateur peut démarrer ou arrêter les scripts de simulation des capteurs et du gateway. Cette fonctionnalité utilise des scripts Python pour gérer les processus en arrière-plan et leurs fichiers PID (**.pid**).

5.2 Interface Frontend



Figure 5.1: Interface Principale

Le frontend est l'interface visuelle du système. Conçue avec **Tailwind CSS** pour un style moderne et réactif, elle permet aux utilisateurs de visualiser l'état du système et de comparer les performances des deux méthodes de chiffrement en temps réel.

5.2.1 Contrôle Centralisé

Le tableau de bord dispose de boutons clairs pour contrôler la simulation des capteurs (mode continu et mode unique) et le lancement ou l'arrêt du gateway. La gestion des fichiers PID permet à l'interface de vérifier le statut de chaque processus et de mettre à jour les boutons et le texte de statut en conséquence. Des messages de notification pop-up fournissent un retour d'information instantané à l'utilisateur sur l'état de ses actions.

5.2.2 Visualisation et Analyse des Performances

L'interface utilise la puissante bibliothèque **Chart.js** pour créer des graphiques dynamiques qui illustrent les données clés du système. Trois graphiques principaux sont affichés :

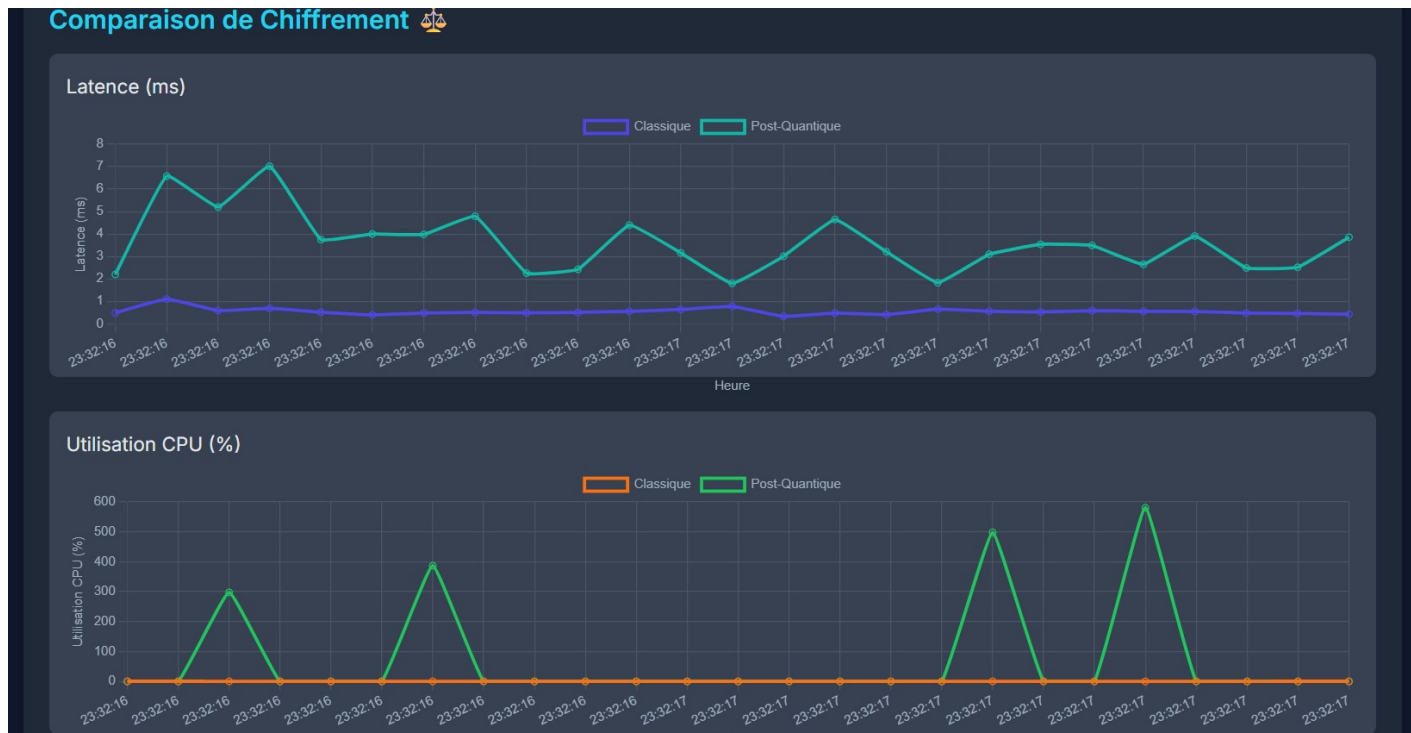


Figure 5.2: Graphiques de Performance (Latence, CPU, Taille)

- **Latence** : Compare la latence en millisecondes des opérations de chiffrement classique et PQC, illustrant le coût temporel de chaque méthode.
- **Utilisation du CPU** : Montre le pourcentage d'utilisation du processeur, révélant la charge de calcul imposée par chaque algorithme de chiffrement.
- **Taille des Données Chiffrées** : Compare la taille des messages chiffrés en octets, mettant en évidence l'impact des algorithmes PQC, souvent plus "volumineux", sur la charge utile des messages.

De plus, un tableau récapitulatif affiche les 50 dernières mesures détaillées, tandis qu'un aperçu des dernières données déchiffrées est présenté sous forme de JSON pour une inspection rapide.

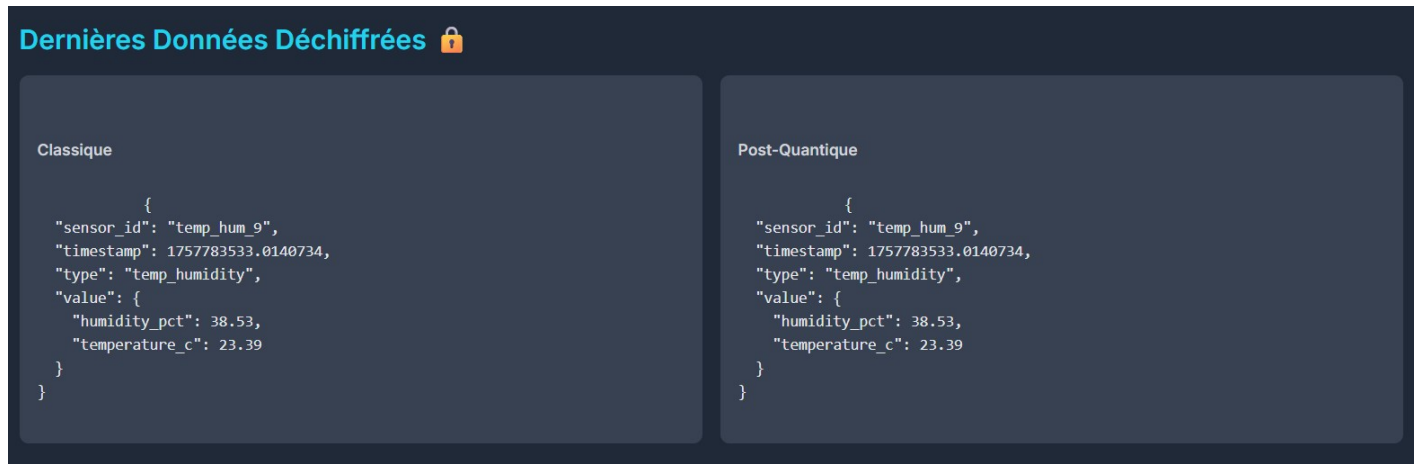


Figure 5.3: Tableau Récapitulatif et Aperçu des Données

Un système de "pause/reprise" permet à l'utilisateur de geler les mises à jour en temps réel pour analyser des données spécifiques sans interférence. Le tout crée une plateforme complète et intuitive pour la surveillance et l'analyse de l'infrastructure de sécurité.

Conclusion et Observations de la Recherche

Ce projet a permis de concevoir, d'implémenter et de tester une solution de sécurité hybride pour l'IoT, intégrant des algorithmes cryptographiques classiques et post-quantiques. Les simulations de capteurs, la passerelle et le serveur central ont fonctionné ensemble pour démontrer un flux de données sécurisé et mesurable. La comparaison des performances entre les deux approches cryptographiques fournit des informations précieuses pour le développement de futures solutions résilientes aux menaces quantiques.

L'analyse des résultats met en évidence un compromis clair. Les algorithmes classiques offrent une performance supérieure en termes de rapidité et d'efficacité, tandis que les algorithmes PQC, bien que plus lents et gourmands en ressources, garantissent une résilience essentielle. Les données ont montré que les messages PQC sont considérablement plus volumineux (environ 9200 octets) que les messages classiques (environ 1075 octets), un facteur critique pour les réseaux à bande passante limitée. L'étude a prouvé la faisabilité de l'approche hybride et sert de base pour une stratégie de migration progressive vers la cryptographie post-quantique dans les environnements IoT.

Glossaire

AES-256-GCM: Un algorithme de chiffrement par bloc symétrique qui assure la confidentialité, l'intégrité et l'authenticité des données.

Chart.js: Une bibliothèque JavaScript qui permet de créer des graphiques pour la visualisation des données sur le web.

Cryptographie hybride: Combinaison de la cryptographie symétrique (pour la vitesse) et asymétrique (pour la distribution sécurisée des clés) afin d'optimiser la sécurité et la performance.

Edge Gateway: Une passerelle de périphérie qui collecte, traite et sécurise les données à la source, près des capteurs.

Flask: Un micro-framework web léger pour Python, utilisé pour le développement du backend du serveur central.

Key Encapsulation Mechanism (KEM): Un type d'algorithme cryptographique asymétrique utilisé pour échanger une clé secrète.

MQTT: Un protocole de messagerie léger, standard ouvert pour les applications de l'Internet des objets (IoT).

Post-Quantum Cryptography (PQC): Des algorithmes cryptographiques conçus pour être sécurisés contre les attaques par des ordinateurs quantiques.

psutil: Une bibliothèque Python pour récupérer des informations sur les processus en cours d'exécution et l'utilisation du système (CPU, mémoire, etc.).

RSA: Un algorithme de cryptographie asymétrique largement utilisé pour le chiffrement et les signatures numériques.

SQLite: Un moteur de base de données relationnelle léger et sans serveur, utilisé pour le stockage local des données.

Tailwind CSS: Un framework CSS "utility-first" pour la création rapide d'interfaces utilisateur modernes.

Liste des Acronymes

AES-GCM	Advanced Encryption Standard - Galois/Counter Mode
CPU	Central Processing Unit
ECDSA	Elliptic Curve Digital Signature Algorithm
IoT	Internet of Things
JSON	JavaScript Object Notation
KEM	Key Encapsulation Mechanism
ML-DSA-65	Module-Lattice-based Digital Signature Algorithm (Dilithium)
ML-KEM-768	Module-Lattice-based Key Encapsulation Mechanism (Kyber)
MQTT	Message Queuing Telemetry Transport
NIST	National Institute of Standards and Technology
PEM	Privacy-Enhanced Mail
PID	Process ID
PQC	Post-Quantum Cryptography
QoS	Quality of Service
REST	Representational State Transfer
RSA	Rivest-Shamir-Adleman